



Imitation Learning

CS 224R

Course reminders

- Start forming final project groups (survey due Weds April 22)
- Homework 1 out today, due Fri April 10
- PyTorch tutorial today at 3:30 pm in Skilling Auditorium

Recap: Terminology

state $\mathbf{s}_t$ - the state of the “world” at time t

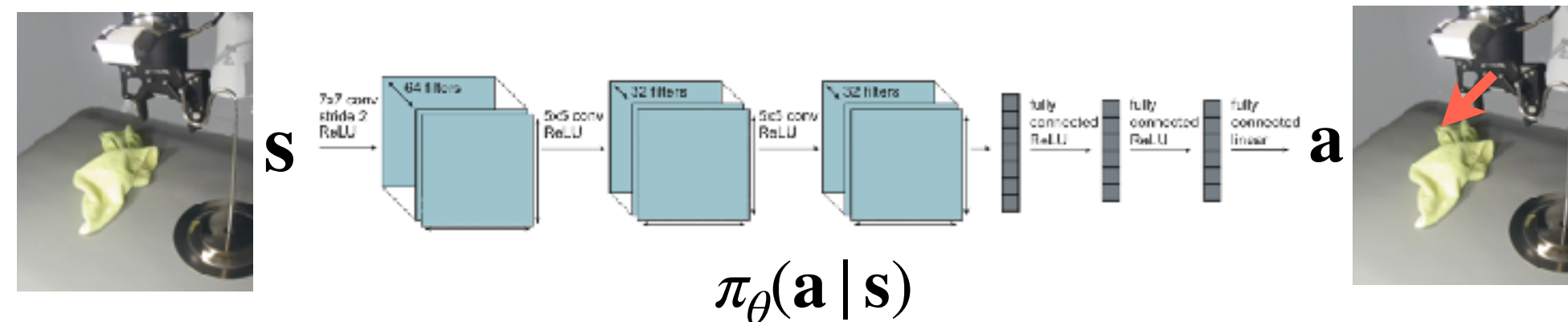
observation $\mathbf{o}_t$ - what the agent observes at time t

action $\mathbf{a}_t$ - the decision taken at time t

trajectory τ - sequence of states/observations and actions

$$(\mathbf{s}_1, \mathbf{a}_1, \mathbf{s}_2, \mathbf{a}_2, \dots, \mathbf{s}_T, \mathbf{a}_T)$$

reward function $r(\mathbf{s}, \mathbf{a})$ - how good is $\mathbf{s}, \mathbf{a}$?



policy $\pi(\mathbf{a}_t | \mathbf{s}_t)$ or $\pi(\mathbf{a}_t | \mathbf{o}_{t-m:t})$ - behavior, usually what we are trying to learn

can be represented using a generative model

Recap: Imitation Learning

Data: Given trajectories collected by an expert

“*demonstrations*” $\mathcal{D} := \{(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T)\}$



(sampled from some unknown policy π_{expert})

Training: For deterministic policy, supervised regression to the expert’s actions

$$\min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{s}, \mathbf{a}) \in \mathcal{D}} \|\mathbf{a} - \hat{\mathbf{a}}\|^2 \quad \text{where } \hat{\mathbf{a}} = \pi_{\theta}(\mathbf{s})$$

The plan for today

Imitation Learning

1. Learning expressive policy distributions
2. Learning from online interventions
3. Time permitting: how to collect demonstrations

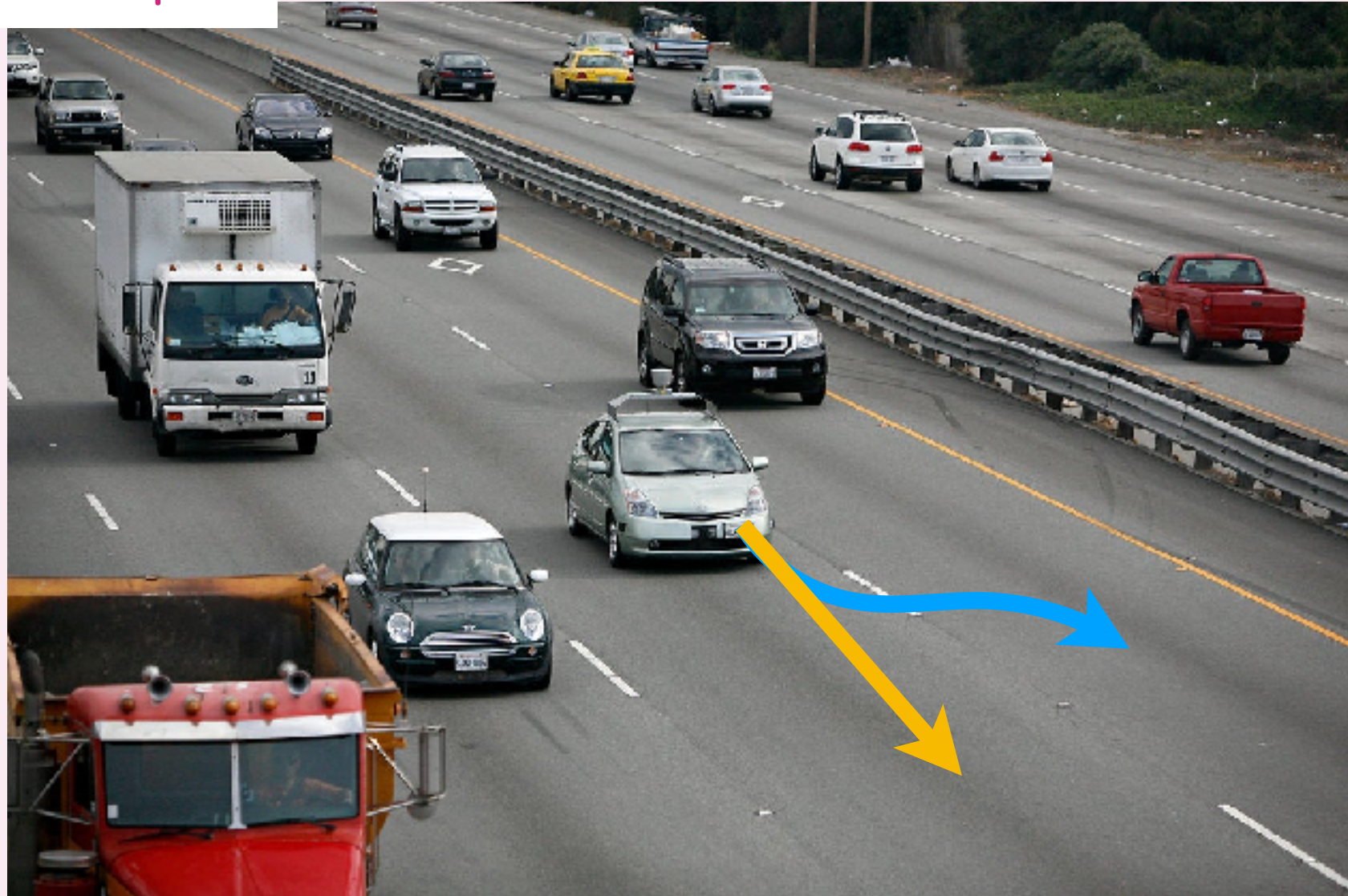
} Topic of homework 1!

Key learning goals:

- how to represent distributions with neural networks
- why expressive distributions matter for imitation learning
- what are compounding errors and how to address them

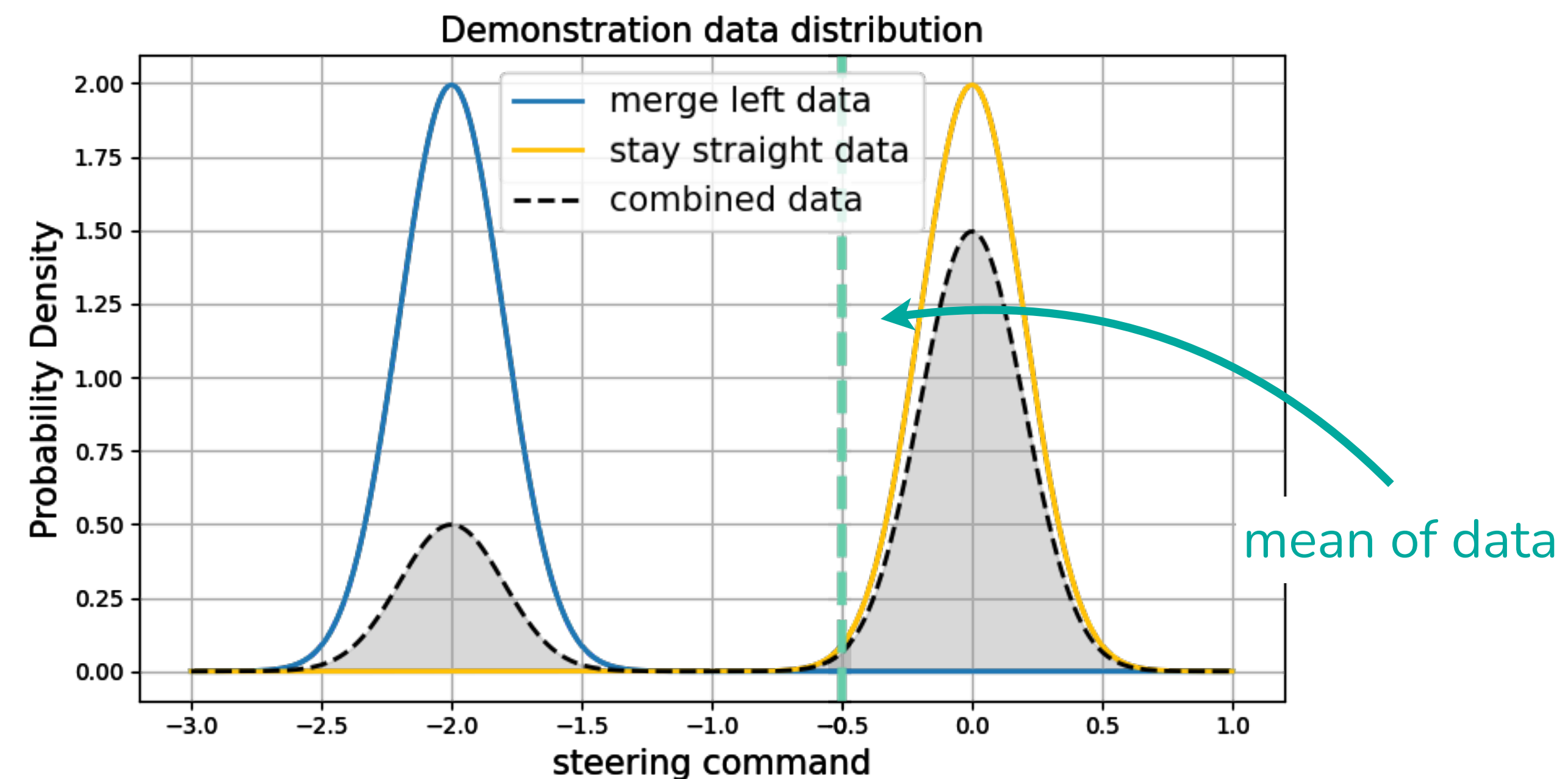
What could go wrong?

Example



- Dataset from human drivers
- Sensor readings + steering commands

Question: what might policy trained with ℓ_2 -regression do?



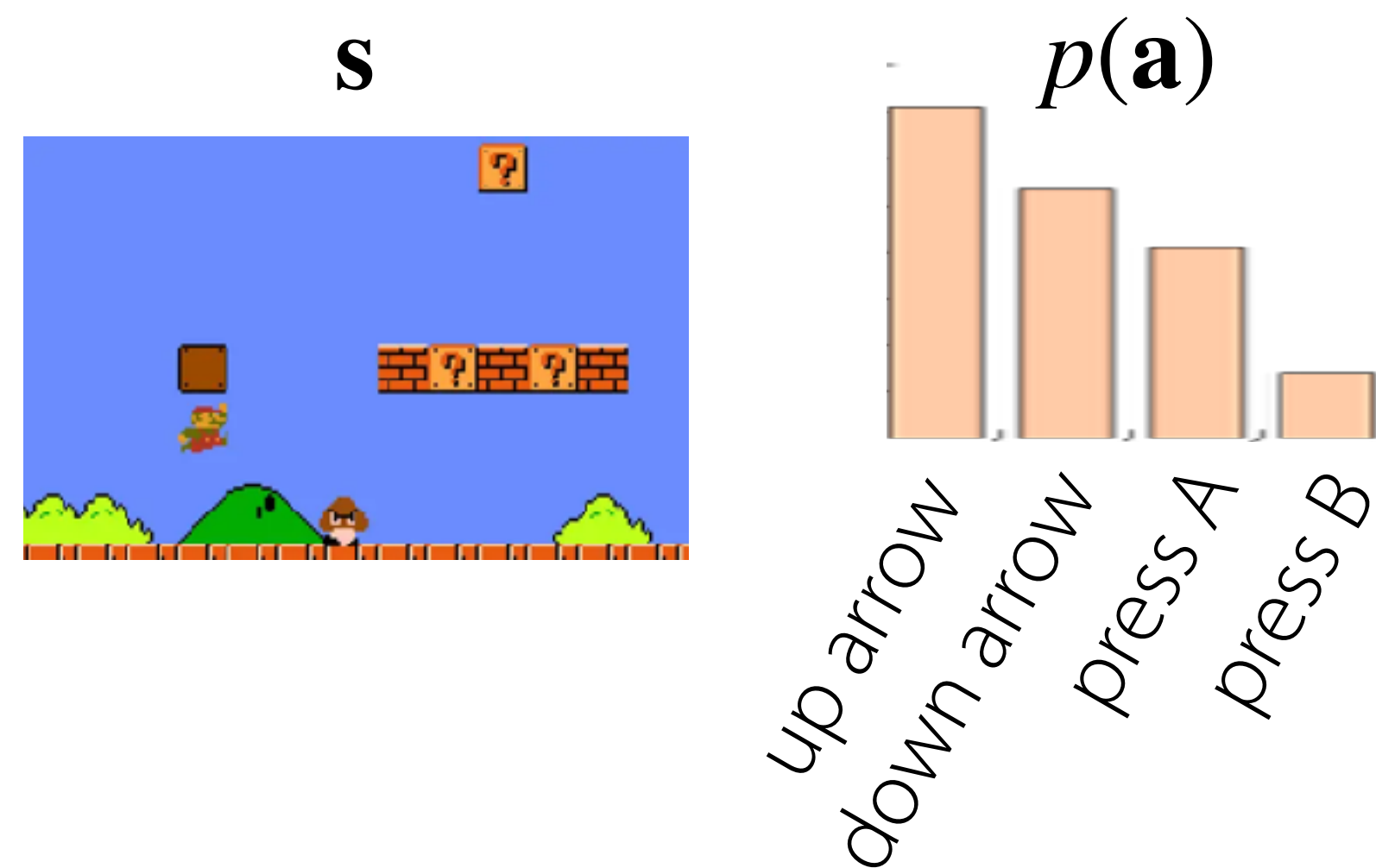
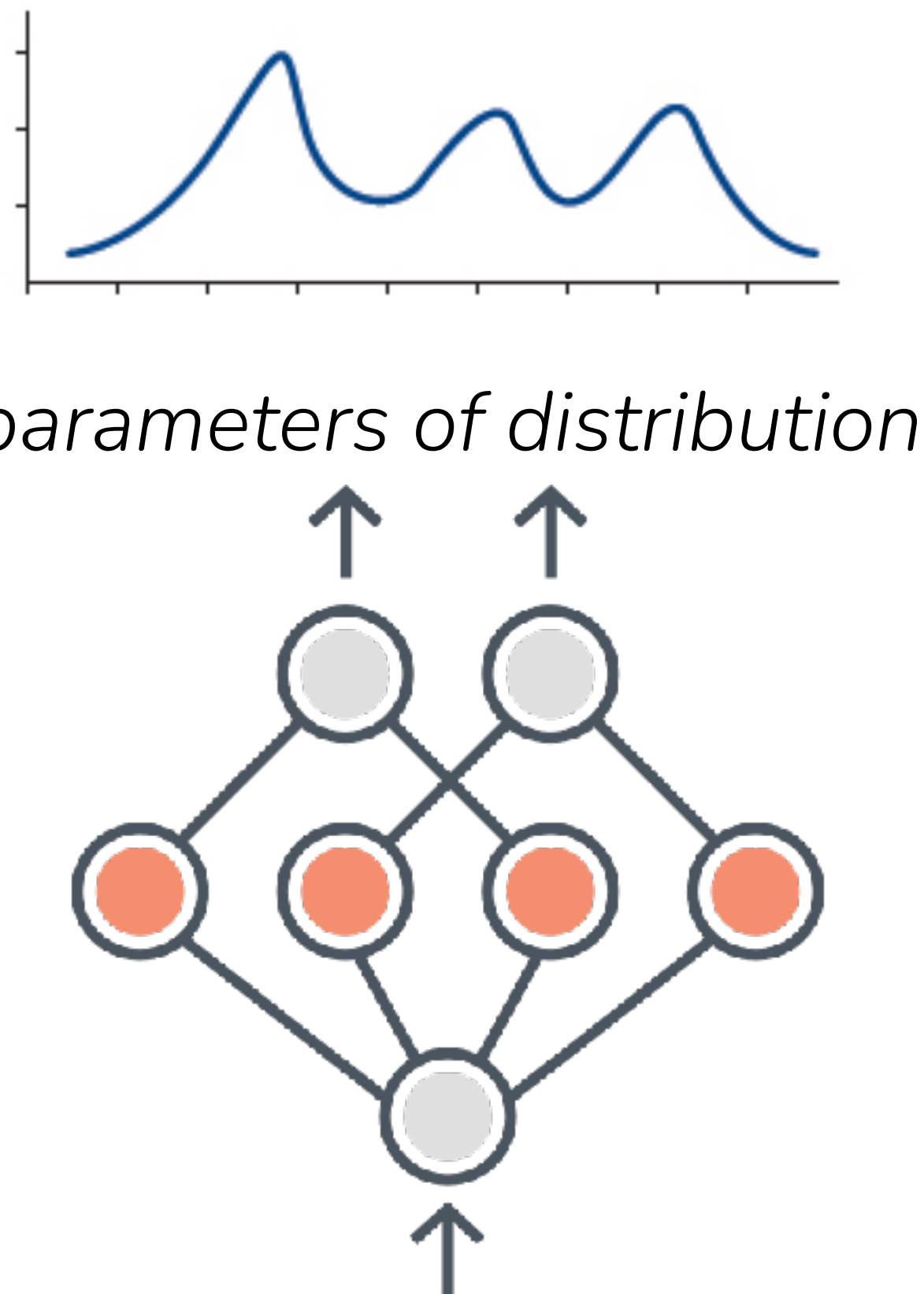
How often does this happen in practice? All the time!

How can we represent more than the mean?

Esp. when data collected by multiple people.

Learning *distributions* with neural networks

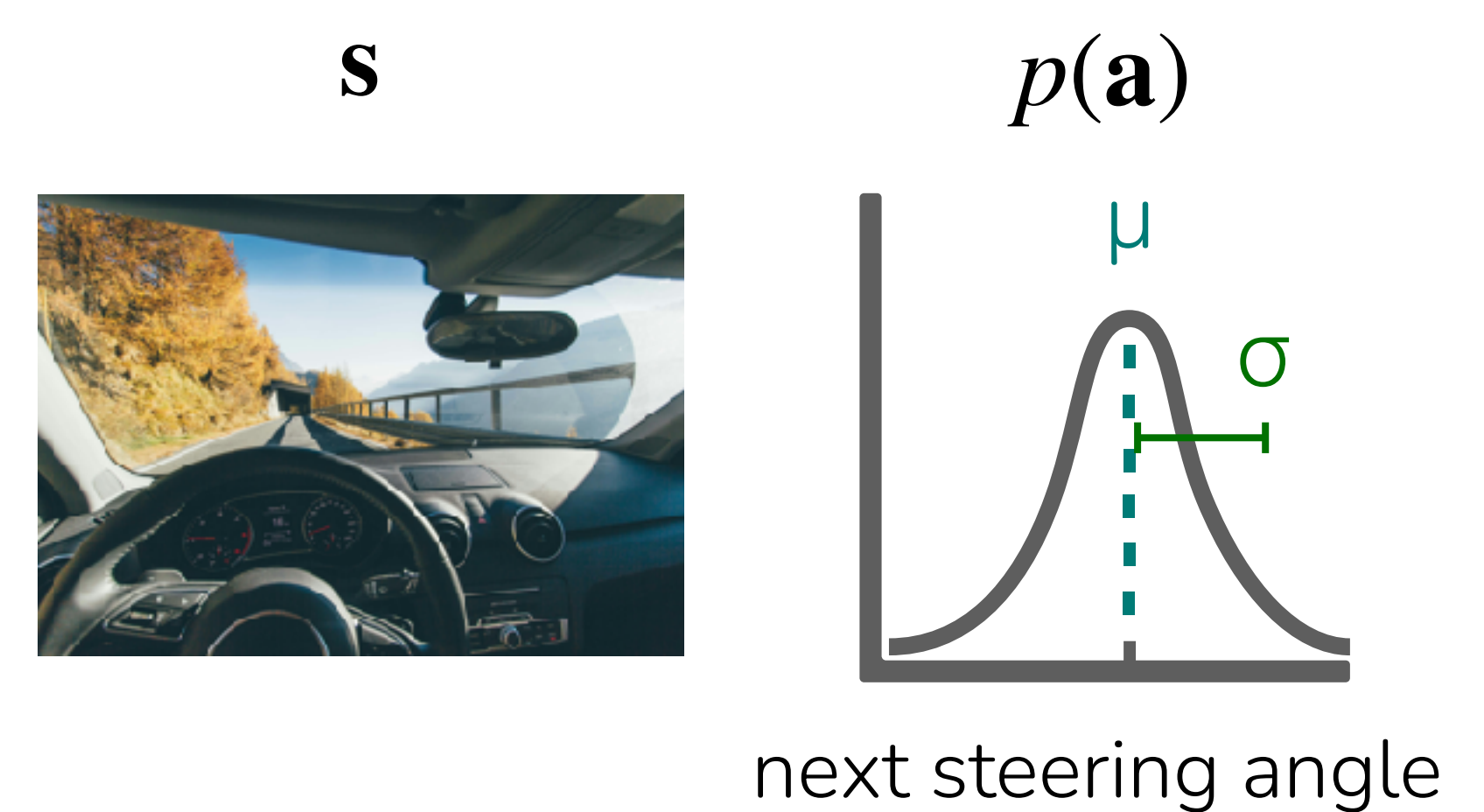
1D discrete actions



Neural net outputs $p(\text{up})$, $p(\text{down})$, ...
represent categorical distribution.

Maximally expressive

Continuous actions



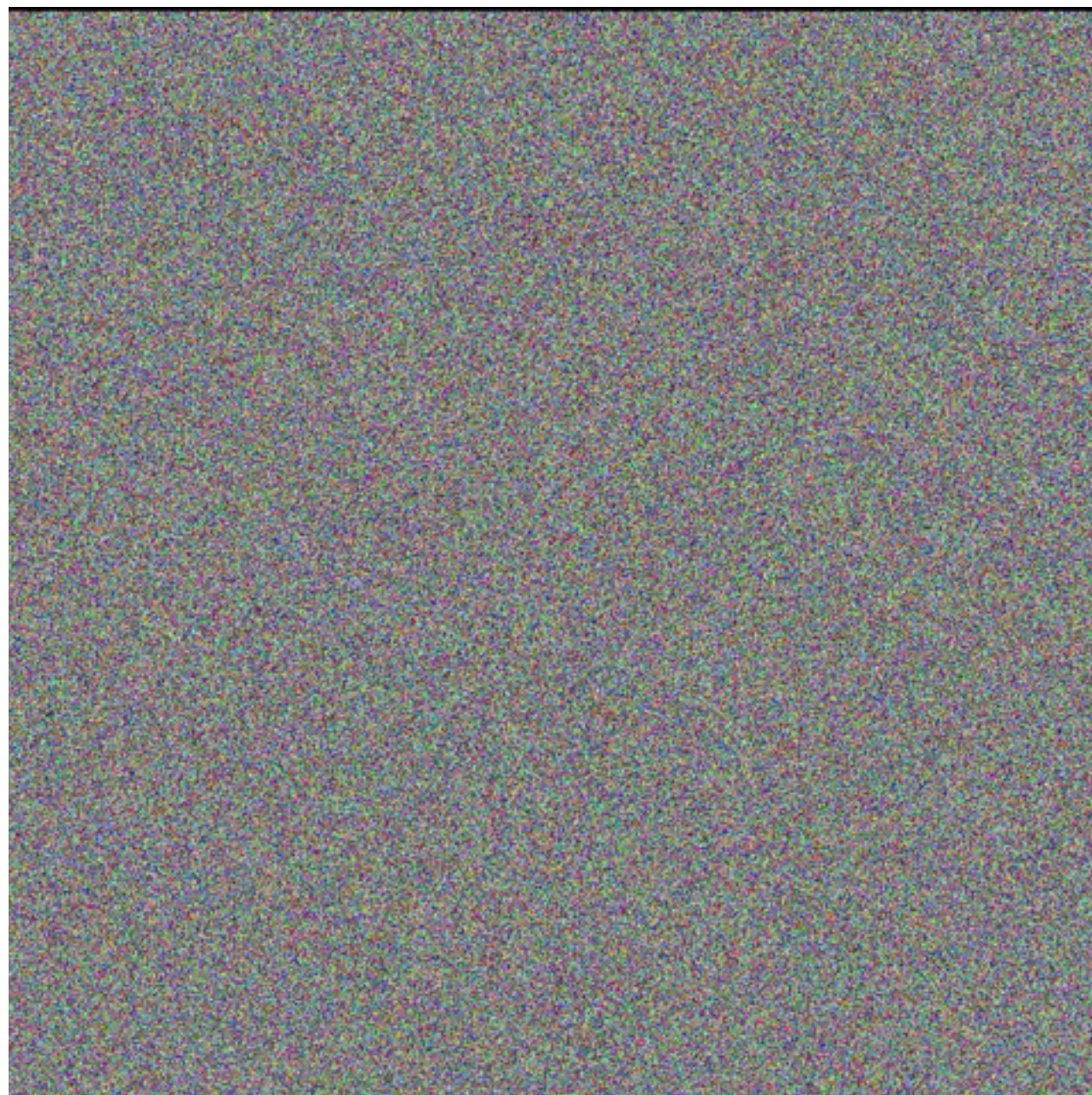
Neural net outputs μ , σ to represent
Gaussian distribution.

Not very expressive!

Learning *distributions* with neural networks

💡 Can we use generative modeling?

image diffusion models



learning $p(\text{image} \mid \text{text description})$

autoregressive models

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

learning $p(\text{next word} \mid \text{words so far})$

Our goal: learning $p(\text{action} \mid \text{observation})$

Imitation learning - version 1

Expressive policies

0. Given demonstrations collected by an expert $\mathcal{D} := \{(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T)\}$

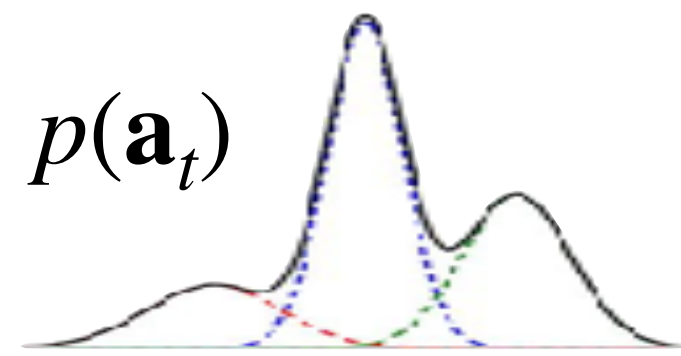
1. Train **generative model** of the expert's actions

$$\min_{\theta} - \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \mathcal{D}} [\log \pi_{\theta}(\mathbf{a} | \mathbf{s})] \quad \text{with expressive distribution } \pi(\cdot | \mathbf{s})$$

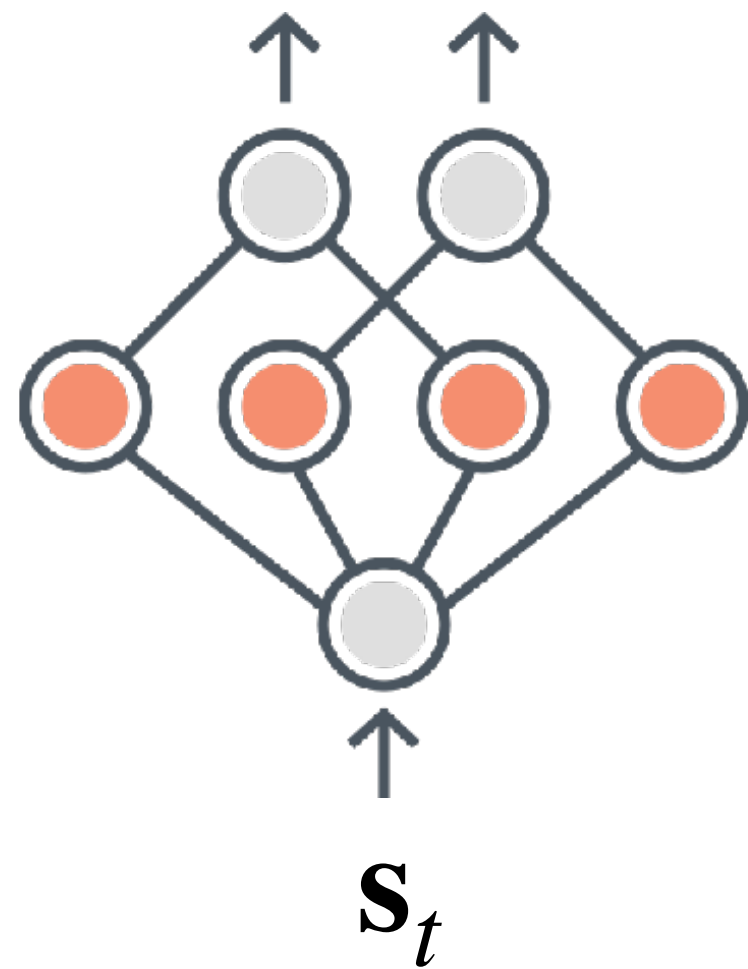
2. Deploy learned policy π_{θ} maximize the log probability of the demo actions under the policy

Generative models for policies (approximating $p(\mathbf{a} | \mathbf{s})$)

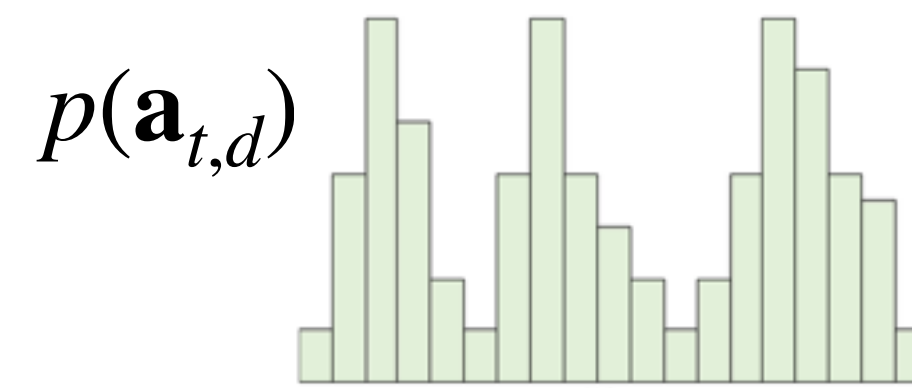
Mixture of Gaussians



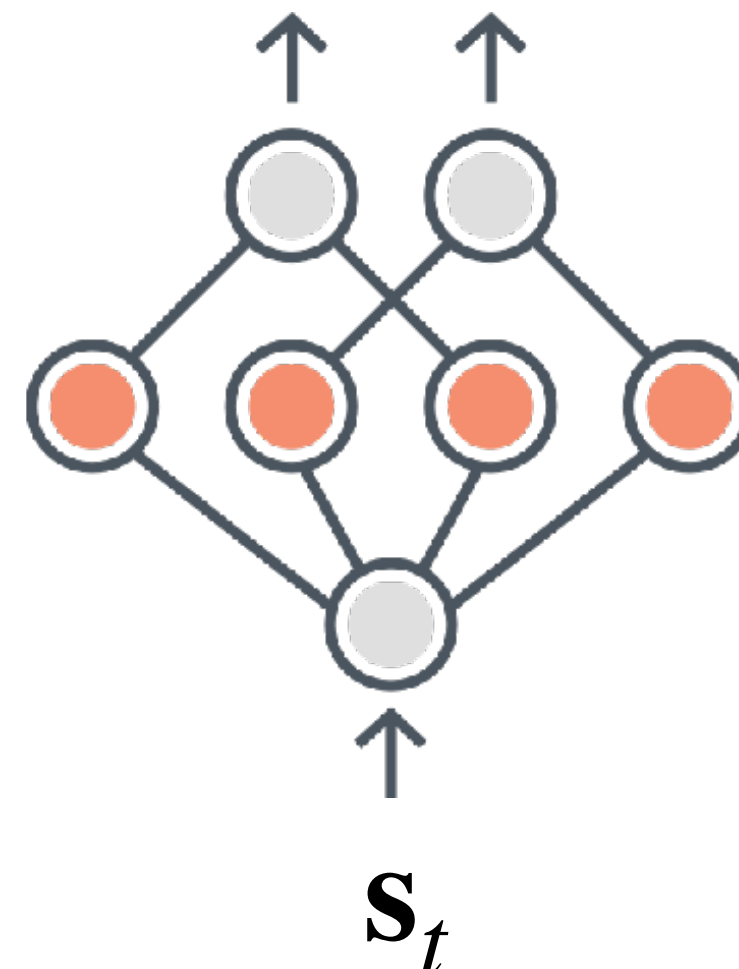
output $\mu_1, \sigma_1, w_1, \mu_2, \sigma_2, w_2, \dots$



Discretize + Autoregressive

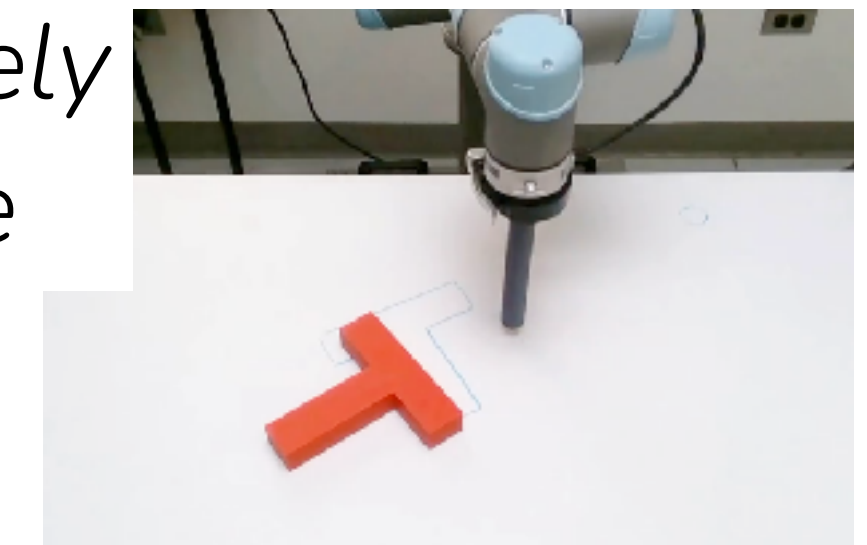


output $p(\mathbf{a}_{t,1}), p(\mathbf{a}_{t,2} | \hat{\mathbf{a}}_{t,1}), p(\mathbf{a}_{t,3} | \hat{\mathbf{a}}_{t,1:2}), \dots$

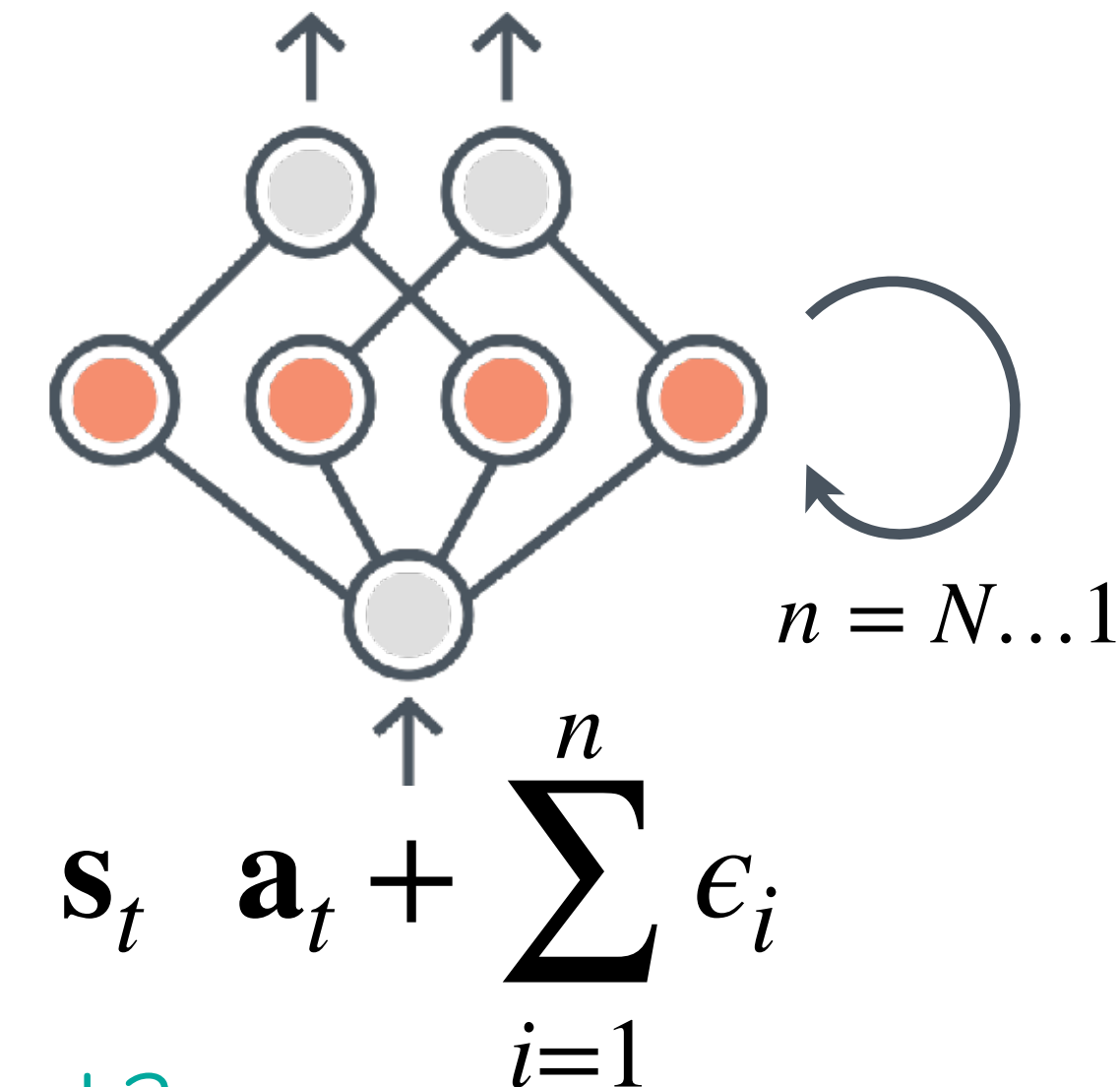


Diffusion

iteratively
denoise



output ϵ_n



Question: how are these different from ℓ_2 loss with larger network?

Important Note: Neural network expressivity is often distinct from distribution expressivity!

How does diffusion work?



Key idea: Can we learn to *transform* samples from Gaussian distribution into samples from the data distribution?

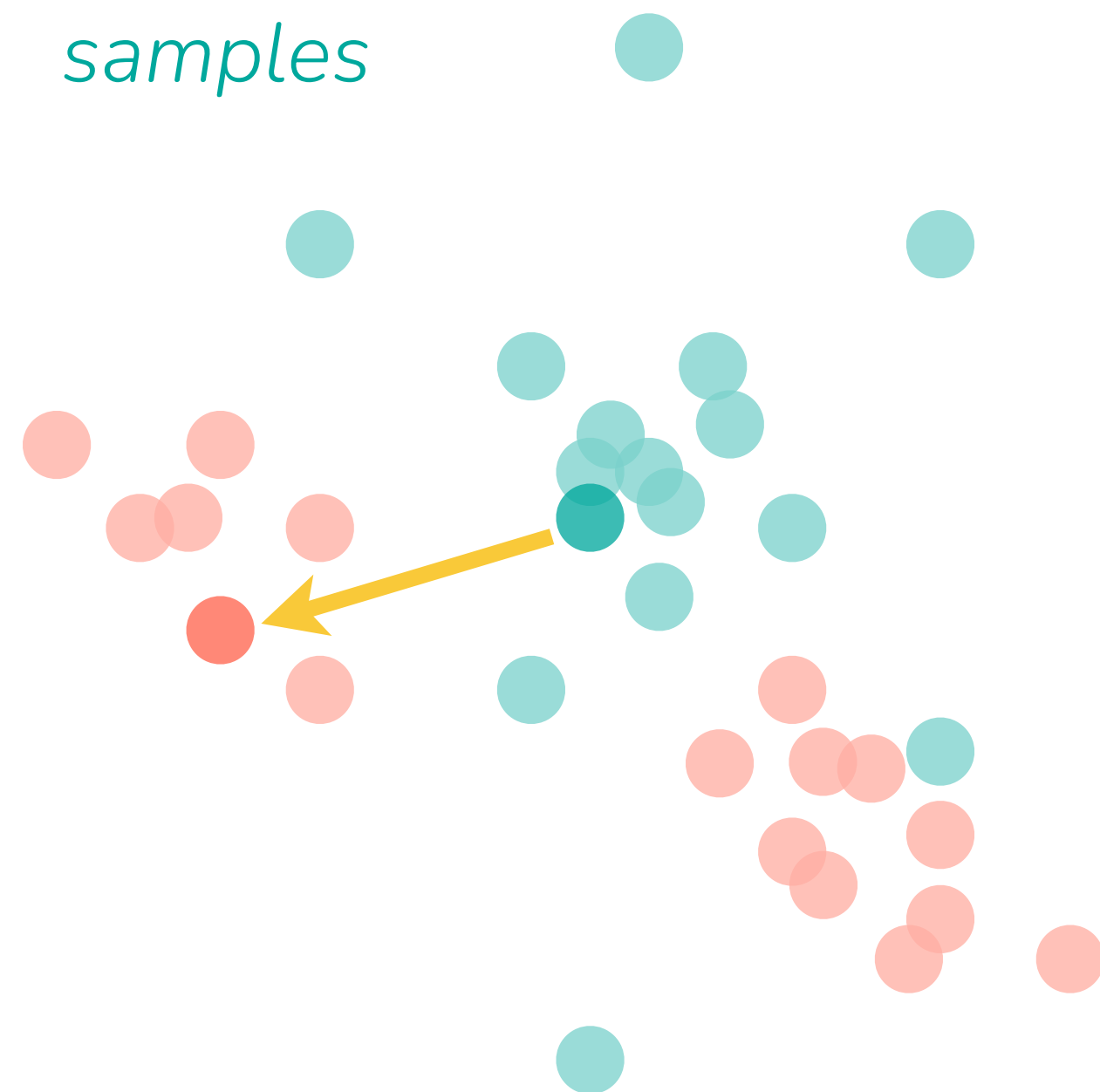


How does diffusion work?

Key idea: Can we learn to *transform* samples from Gaussian distribution into samples from the data distribution?

Let's look at a kind of diffusion called flow matching.

Gaussian noise
samples



Examples in dataset

Idea 0:

1. Randomly sample datapoint x_i and noise $\epsilon \sim \mathcal{N}(0, I)$
2. Update model to regress from noise to datapoint $f_\theta(\epsilon) \rightarrow x_i$
3. Repeat

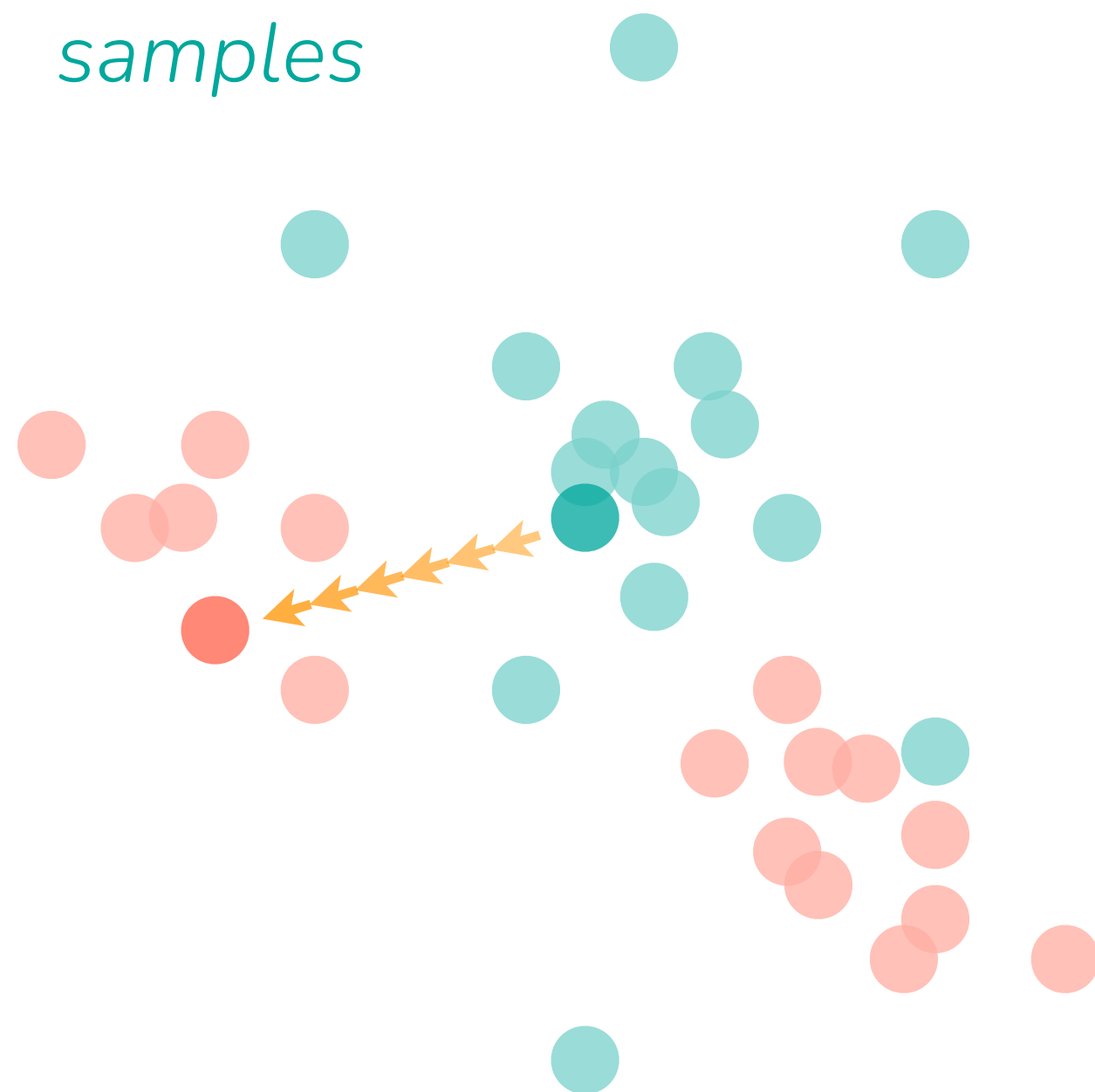
Key challenge: the model has no reason to use the noise to map to different datapoints!

How does diffusion work?

Key idea: Can we learn to *transform* samples from Gaussian distribution into samples from the data distribution?

Let's look at a kind of diffusion called *flow matching*.

Gaussian noise
samples



Examples in dataset

Instead: Learn to predict *vector field* v_θ , for iterative denoising

Training time loop:

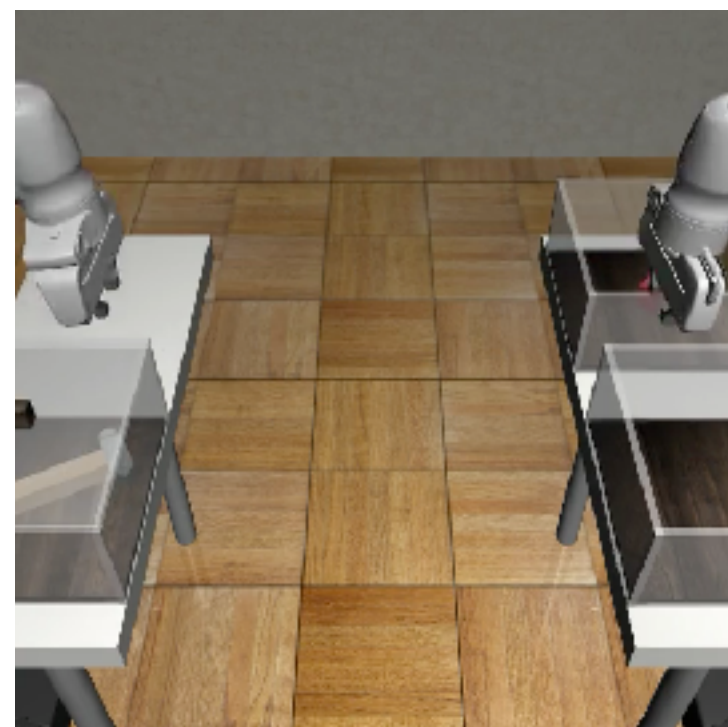
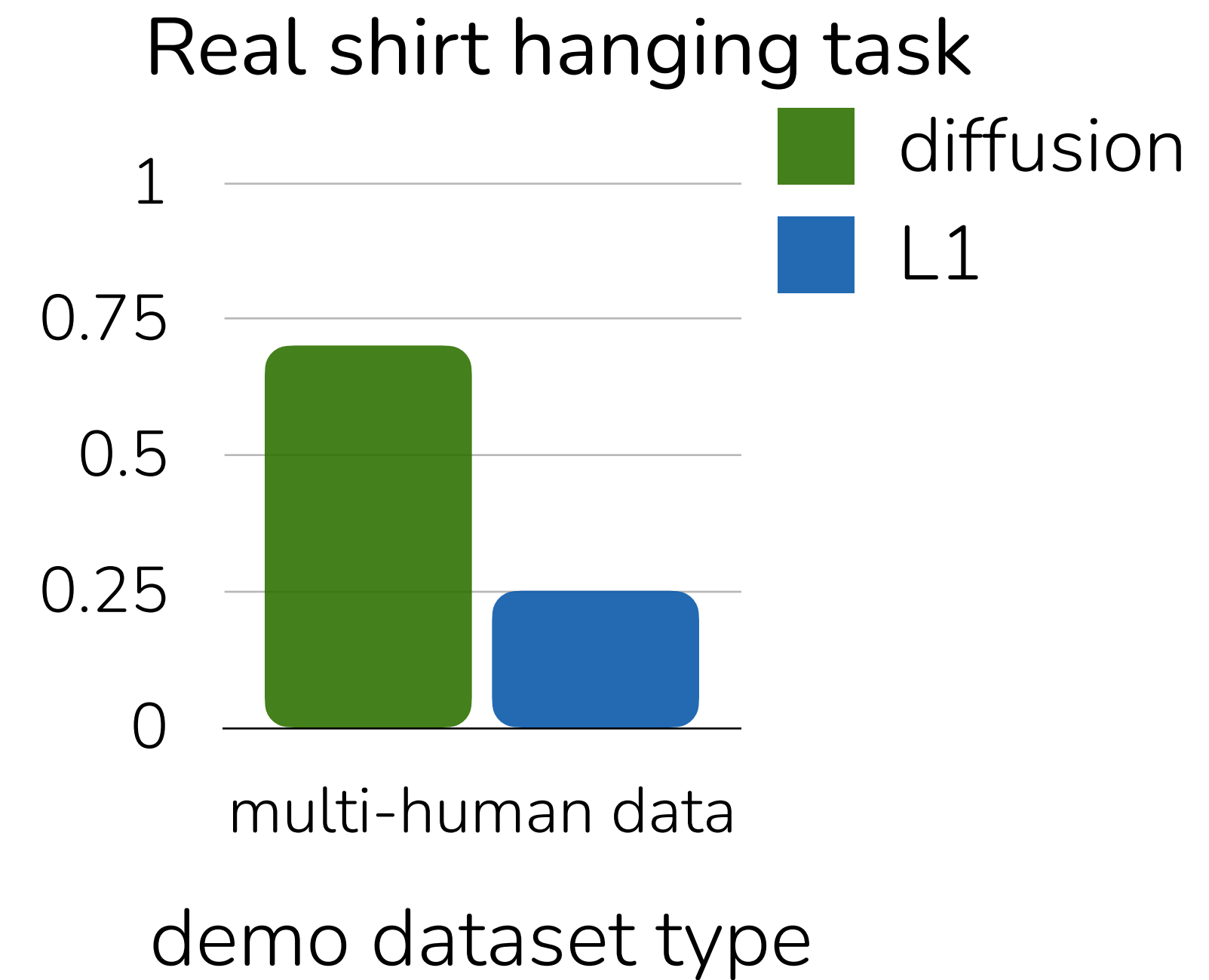
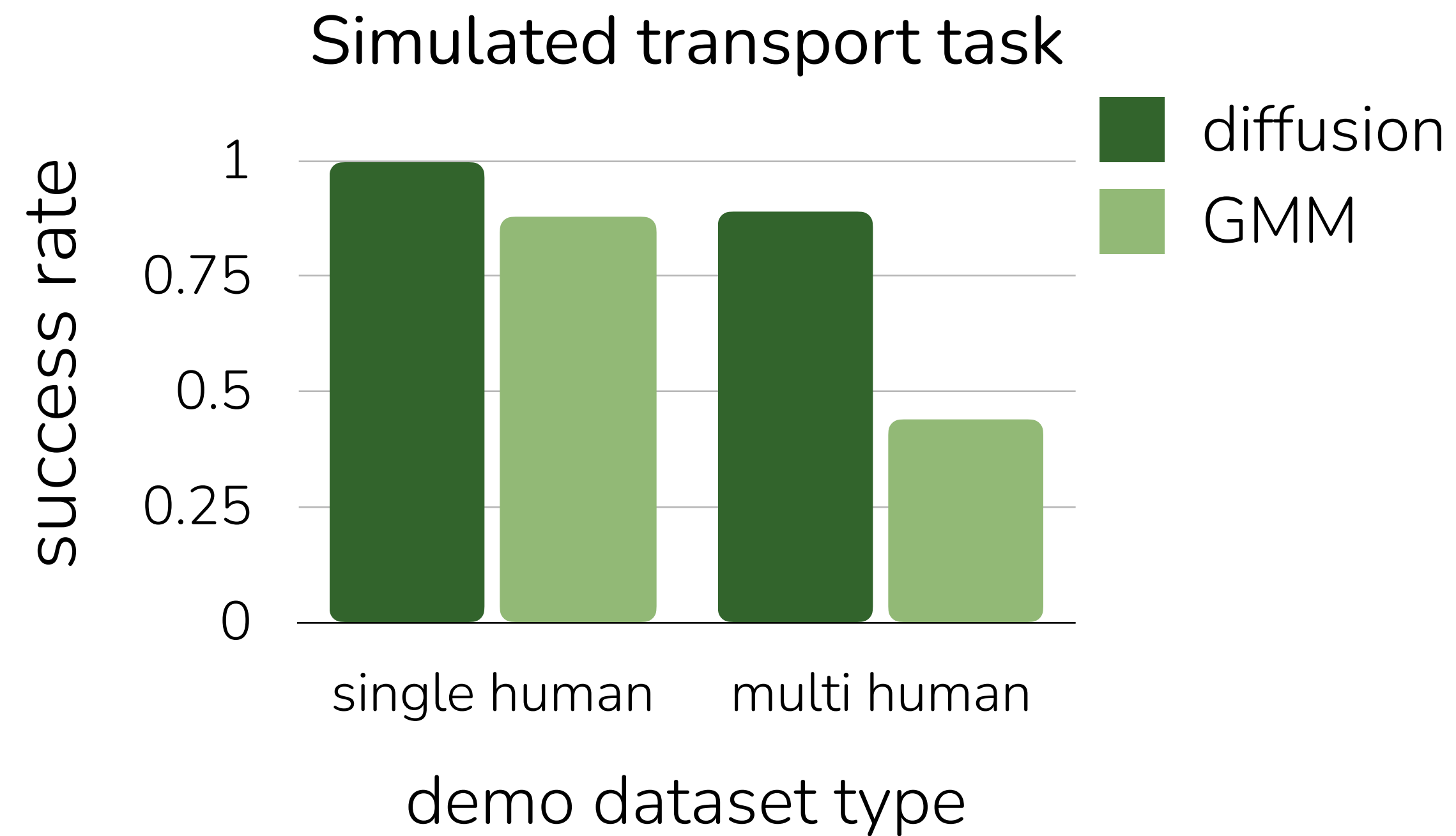
1. Randomly sample datapoint $x_1 \sim D$ and noise $x_0 \sim \mathcal{N}(0, I)$
2. Sample $t \sim p(t)$ (e.g. $p(t) = \text{Unif}[0, 1]$)
3. Linearly interpolate $x_t = tx_1 + (1 - t)x_0$
4. Minimize $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$

Test time

1. Randomly sample noise $x_0 \sim \mathcal{N}(0, I)$
2. For $t \in \{0, \delta, 2\delta, 3\delta, \dots, 1 - \delta\}$,
 1. $x_{t+\delta} \leftarrow x_t + v(x_t, t)\delta$ (integrate the ODE with the Euler method)
3. Return x_1

For imitation learning: learn *conditional* vector field for denoising actions a_t (conditioned on state s_t)

Imitation learning - version 1 vs version 0



Robotics: Imitation learning + expressive policies

Physical Intelligence $\pi_{0.6}$



NVIDIA Gr00t

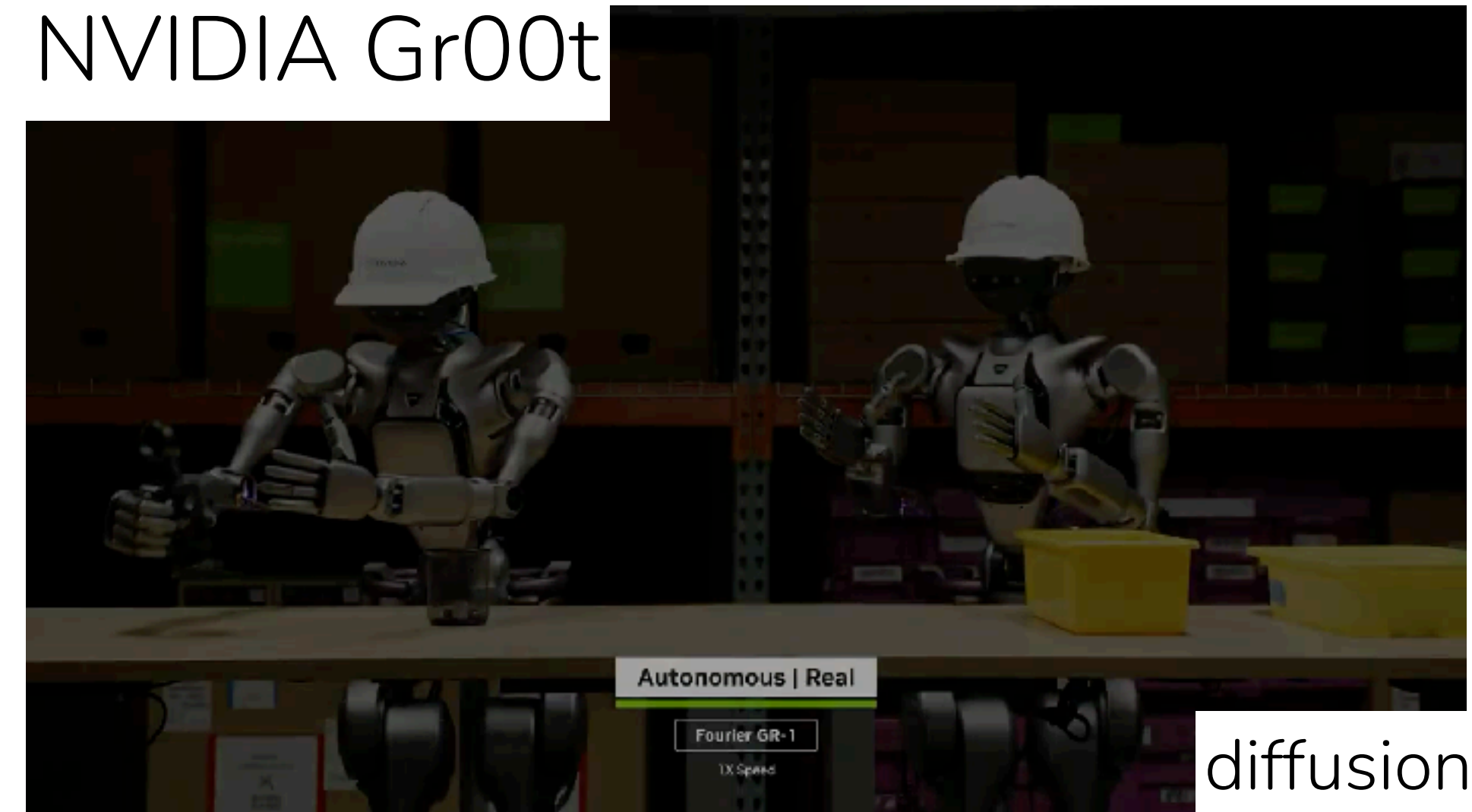
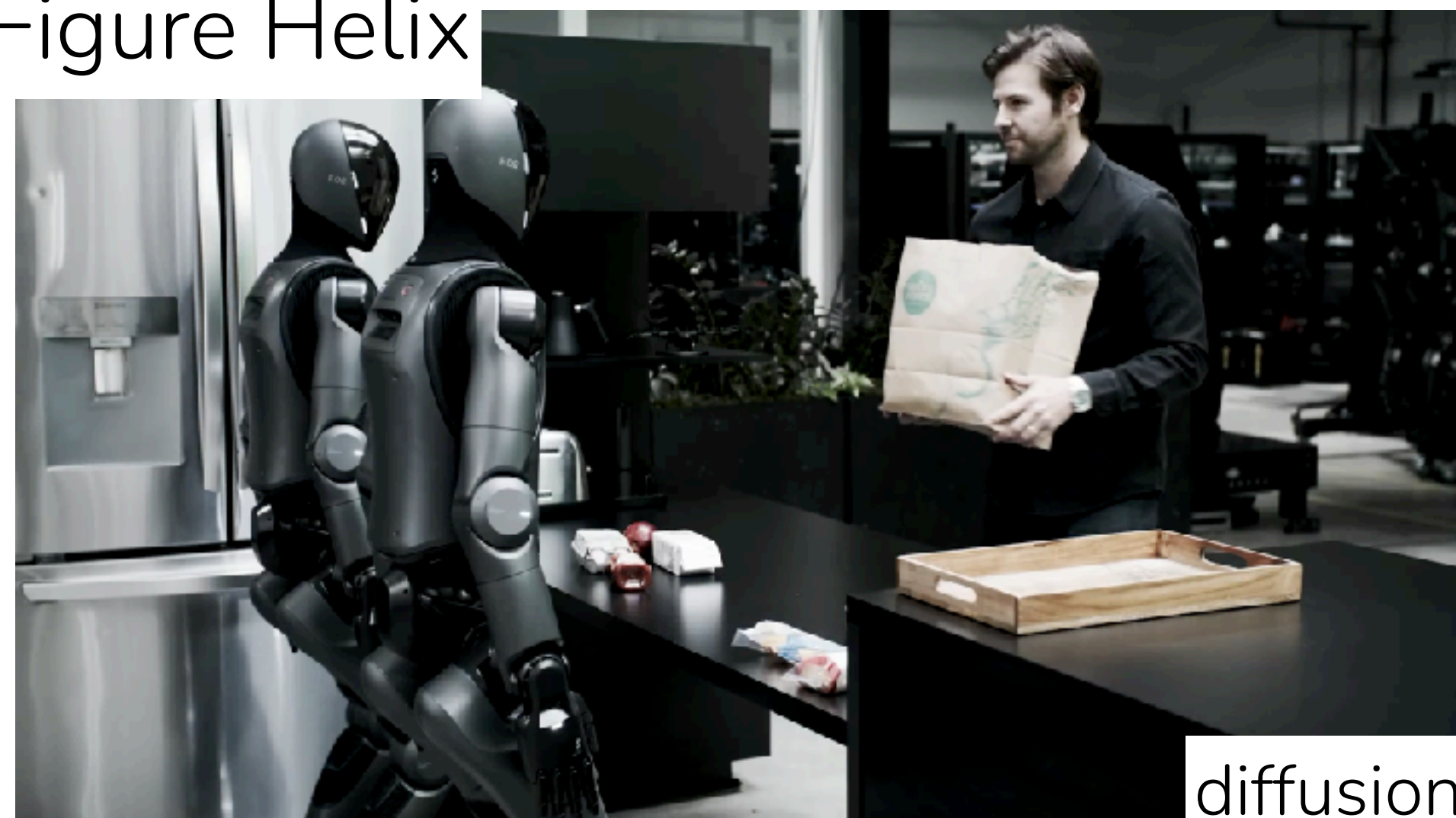


Figure Helix

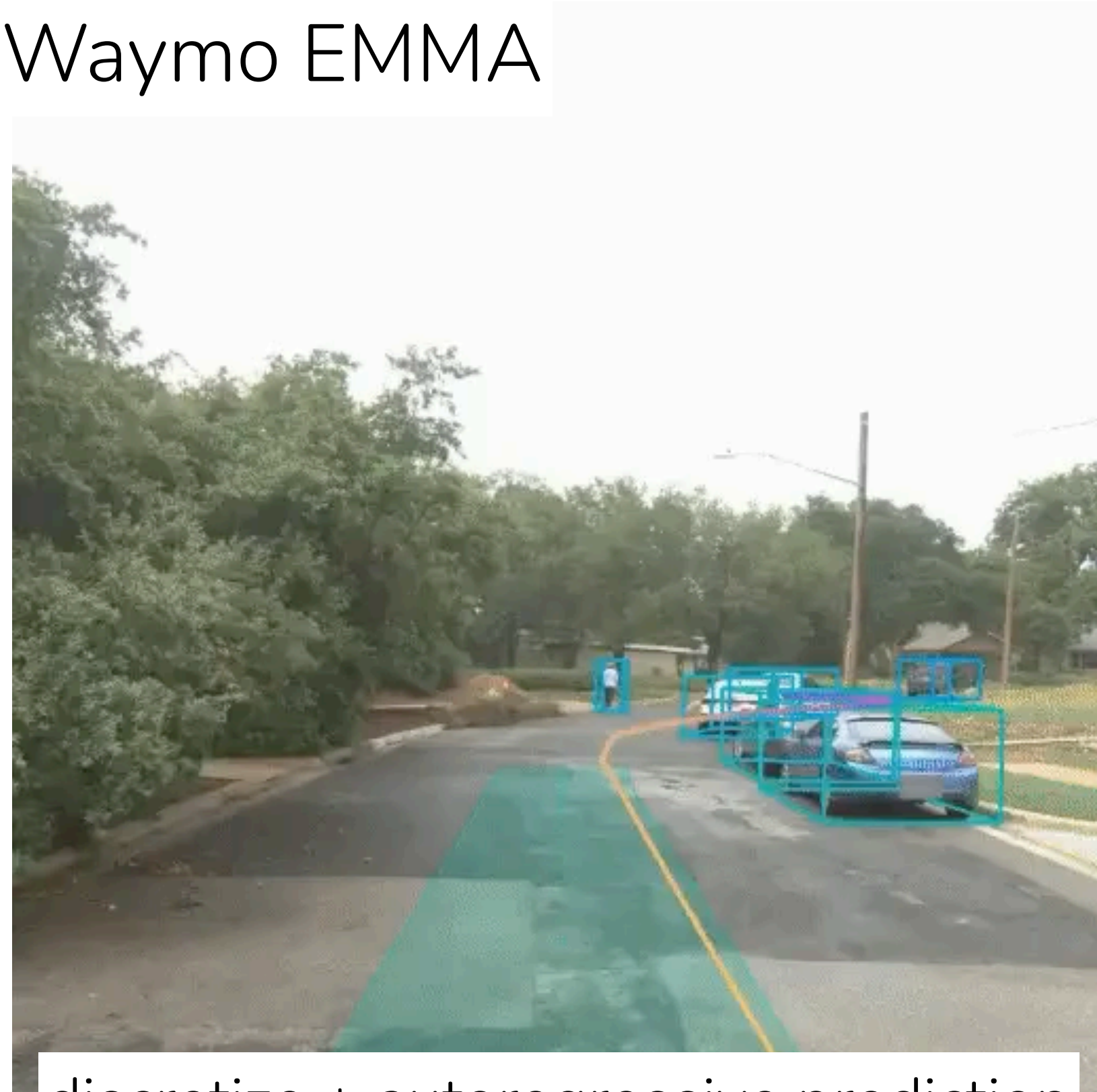


OpenVLA



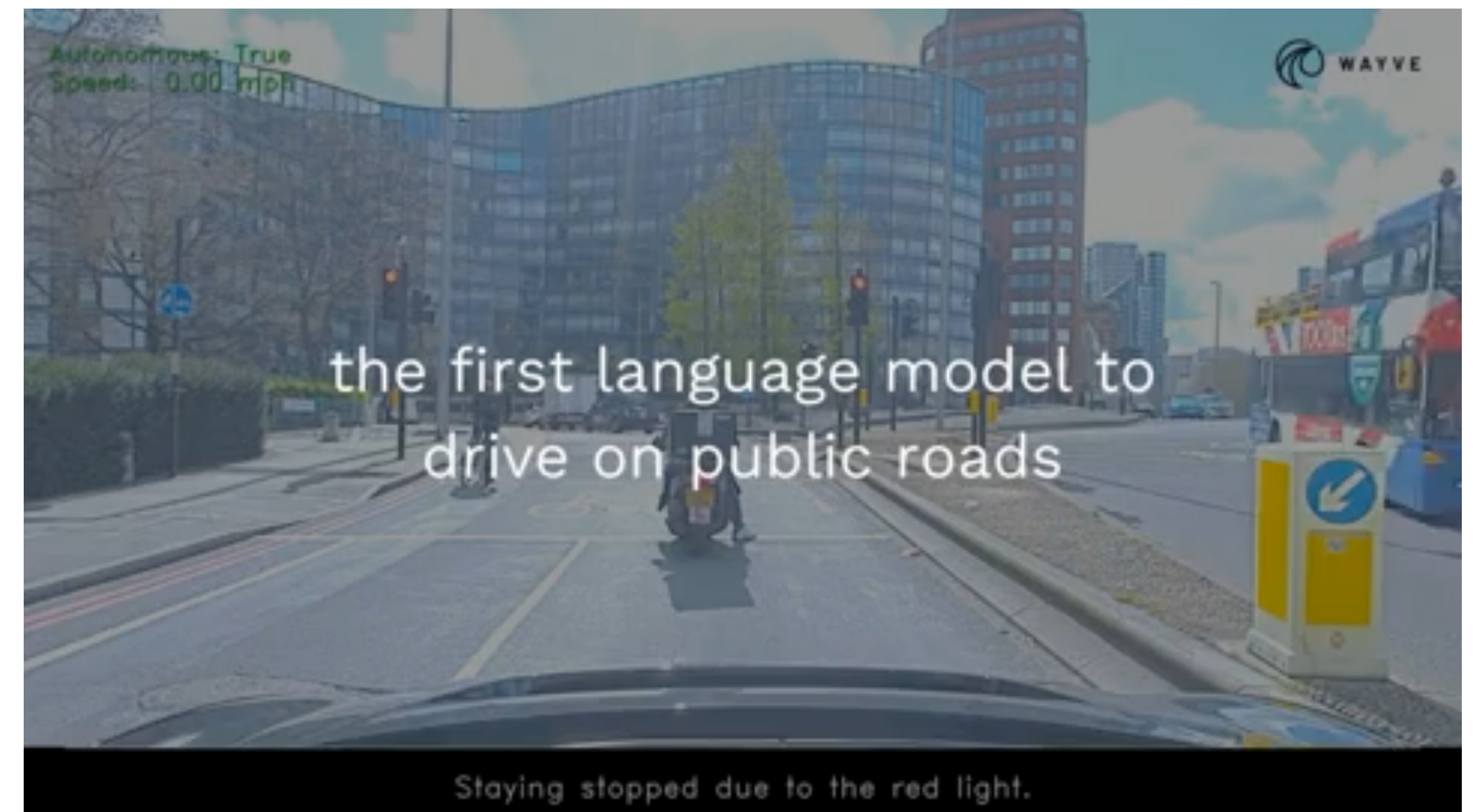
Autonomous driving: Imitation learning + expressive policies

Waymo EMMA



discretize + autoregressive prediction

Wayve LINGO-2



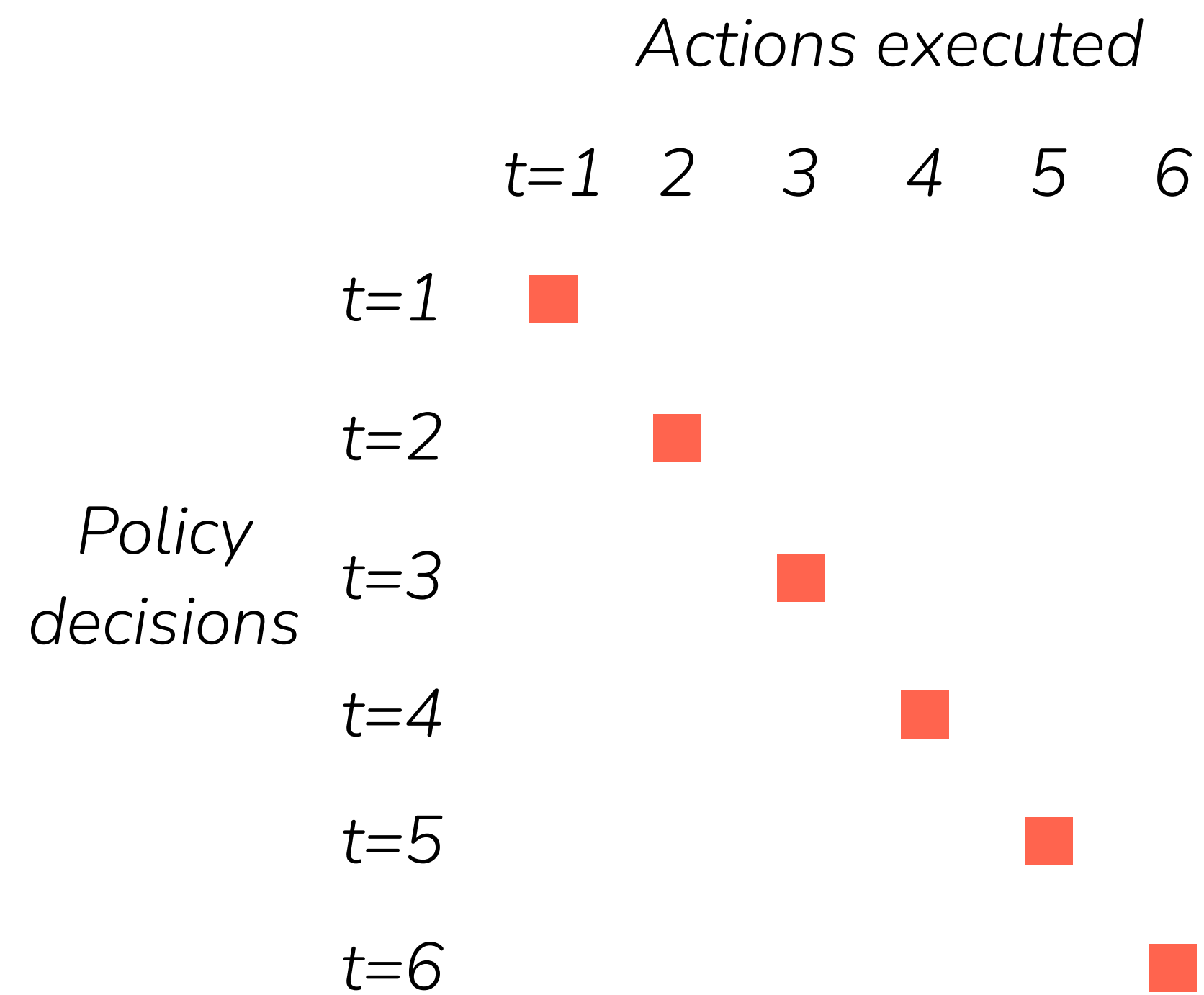
discretize + autoregressive prediction

One more trick

So far: learning policy $\pi(a_t | s_t)$

—> If timestep frequency is 50 Hz, then making new decisions every 20 ms.

Alternative: predict a chunk of actions $a_{t:t+k}$ based on state s_t and make new decisions less frequently (every k steps).



Fully closed loop policy $\pi(a_t | s_t)$



Policy $\pi(a_{t:t+3} | s_t)$ with action chunking
(chunks executed open loop)

Why?

1. It often works way better
2. Allows more computation time for policy inference

Additional references on action chunking

Original papers to introduce action chunking

Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (<https://tonyzhaozh.github.io/aloha/>)

Diffusion Policy: Visuomotor Policy Learning via Action Diffusion (<https://diffusion-policy.cs.columbia.edu/>)

Further analysis / discussion

Action Chunking and Exploratory Data Collection Yield Exponential Improvements in Behavior Cloning for Continuous Control (<https://arxiv.org/pdf/2507.09061>)

Bidirectional Decoding: Improving Action Chunking via Guided Test-Time Sampling (<https://arxiv.org/pdf/2408.17355>)

Real-Time Execution of Action Chunking Flow Policies (<https://arxiv.org/pdf/2506.07339>)

Summary so far

Data from one consistent demonstrator



Unimodal policy distribution is enough

Multimodal data, e.g. from multiple demonstrators



Need expressive generative model for policy

Summary so far

- **Key idea:** Train expressive policy class via generative modeling on dataset of demonstrations.
- Algorithm is fully *offline*

Definitions.

offline: using only an existing dataset, no new data from learned policy

online: using new data from learned policy

- + no need for data from policy (online data can be unsafe, expensive to collect)
- + no need to define a reward function
- may need **a lot** of data for reliable performance

The plan for today

Imitation Learning

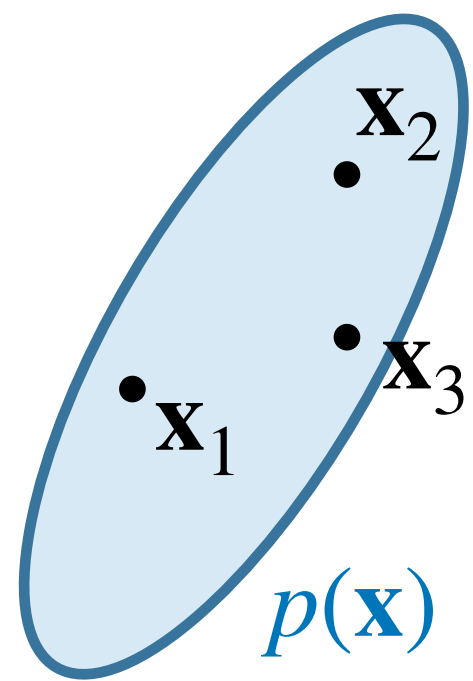
1. Learning expressive policy distributions
2. Learning from online interventions
3. Time permitting: how to collect demonstrations

} Topic of homework 1!

What can go wrong in imitation learning?

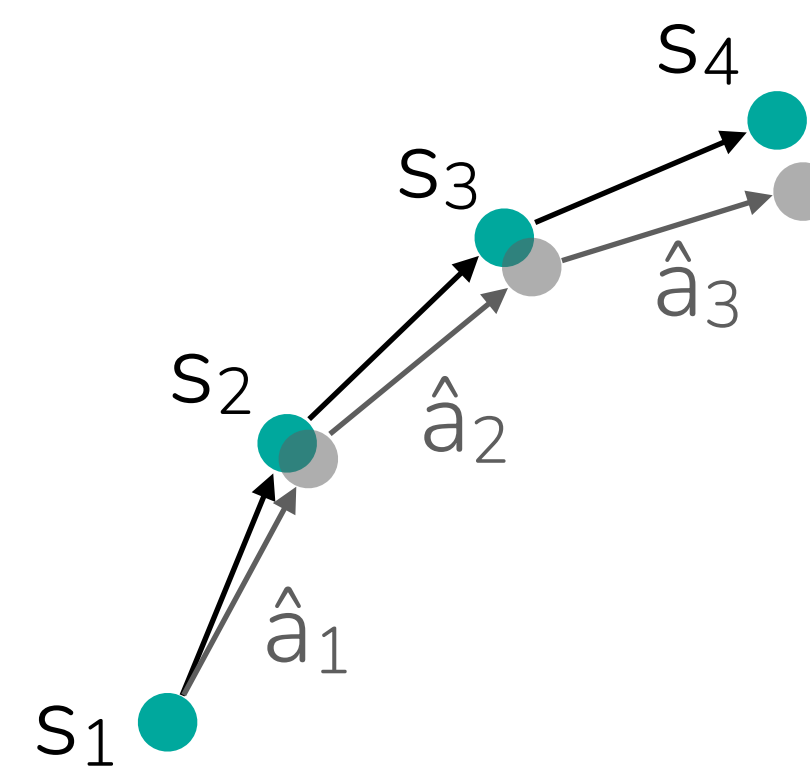
Compounding errors

Supervised learning



Inputs independent
of predicted labels $\hat{\mathbf{y}}$

Supervised learning of behavior



Predicted actions affect
next state.

Errors can lead to drift
away from the data
distribution!

Errors can then compound!

$$\underline{p_{expert}(\mathbf{s})} \neq \underline{p_{\pi}(\mathbf{s})}$$

states visited
by expert

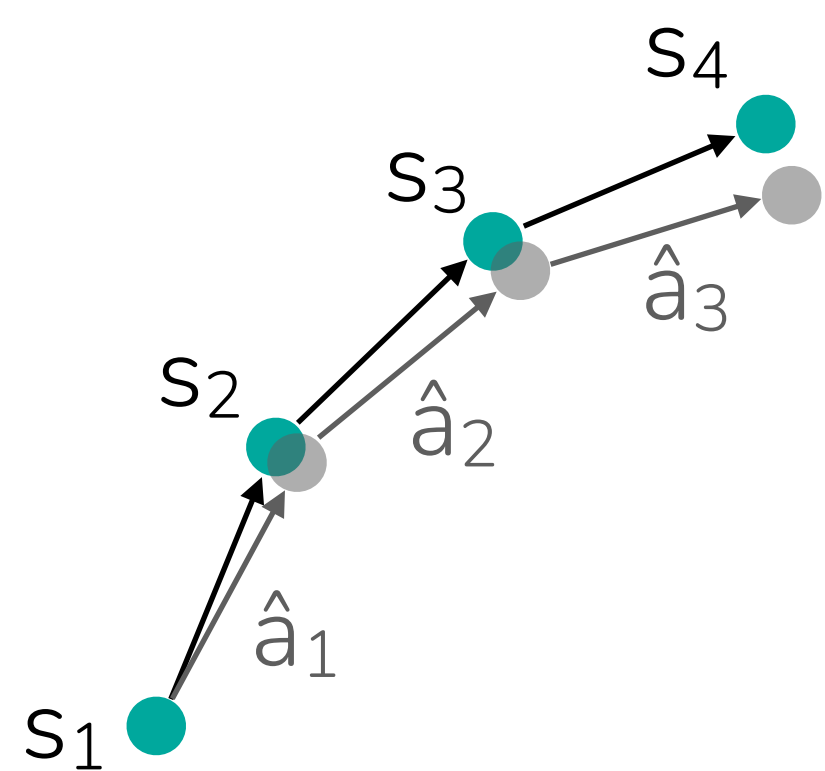
states visited by
learned policy π

“covariate shift”

What can go wrong in imitation learning?

Compounding errors

Supervised learning of behavior



Predicted actions affect next state.

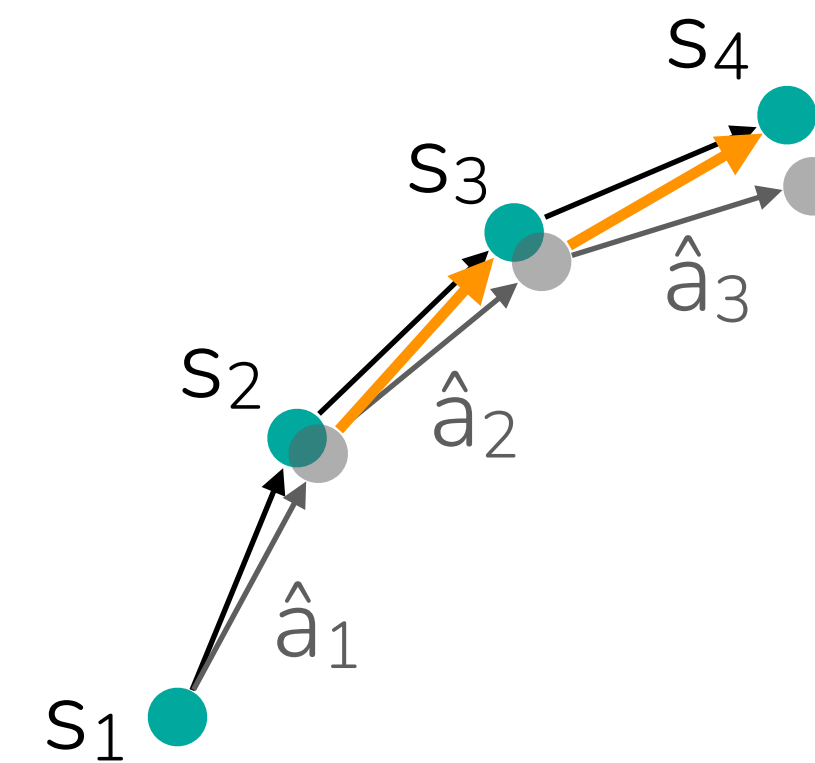
Errors can lead to drift away from the data distribution!

Errors can then compound!

$$p_{expert}(s) \neq p_{\pi}(s)$$

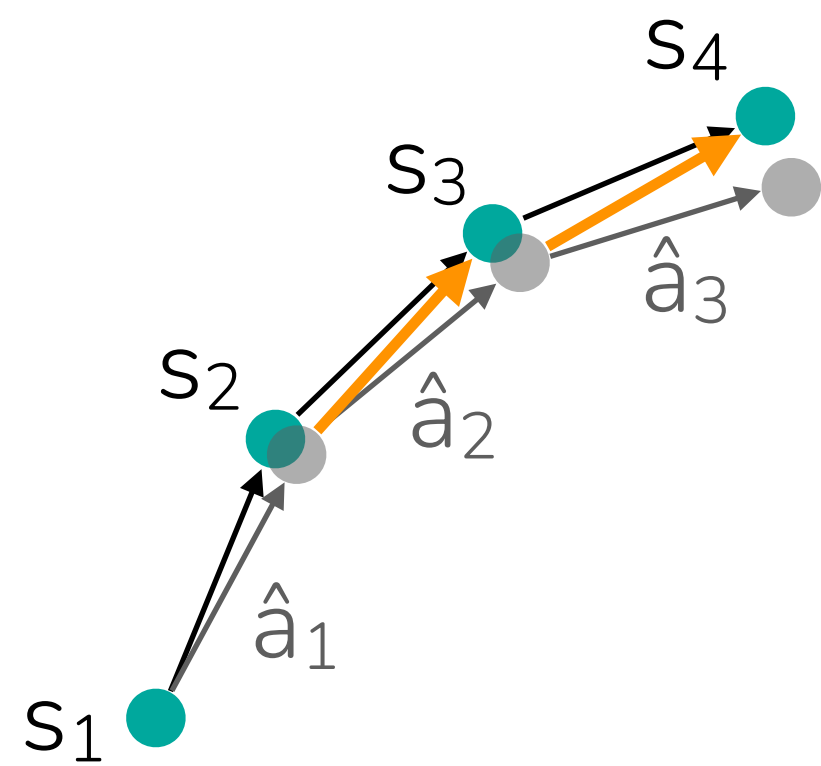
Solutions?

1. Collect A LOT of demo data & hope for the best.
2. Collect **corrective behavior data**



Addressing Compounding Errors with Interventions

Collect **corrective behavior data**



1. Roll out learned policy $\pi_\theta: \mathbf{s}'_1, \hat{\mathbf{a}}_1, \dots, \mathbf{s}'_T$
2. Query expert action at visited states $\mathbf{a}^* \sim \pi_{expert}(\cdot | \mathbf{s}')$
3. Aggregate corrections with existing data $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}', \mathbf{a}^*)\}$
4. Update policy $\min_{\theta} \mathcal{L}(\pi_\theta, \mathcal{D})$

“dataset aggregation” (DAgger)

+ data-efficient way to learn from an expert

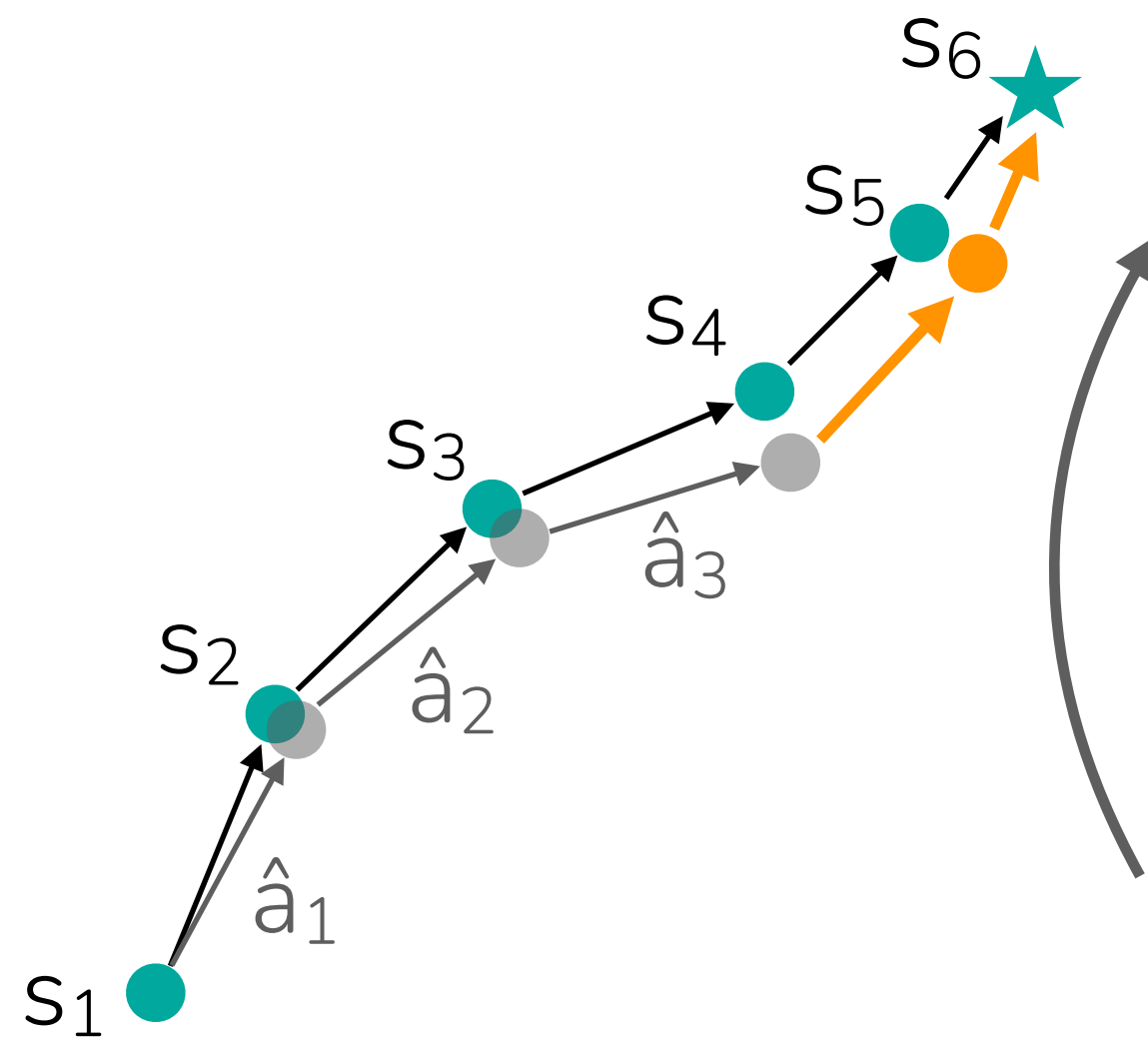
- can be challenging to query expert when agent has control

Is there another way to collect corrective data?



Addressing Compounding Errors with Interventions

Collect **corrective behavior data** while *taking full control*



1. Start to roll-out learned policy $\pi_\theta: \mathbf{s}'_1, \hat{\mathbf{a}}_1, \dots, \mathbf{s}'_t$
2. Expert intervenes at time t when policy makes mistake
3. Expert provides (partial) demonstration $\mathbf{s}'_t, \mathbf{a}_t^*, \dots, \mathbf{s}'_T$
4. Aggregate new demos with existing data $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}'_i, \mathbf{a}_i^*)\}; i \geq t$
5. Update policy $\min_{\theta} \mathcal{L}(\pi_\theta, \mathcal{D})$

“human gated DAgger”

+ (much) more practical interface for providing corrections

- can be hard to catch mistakes quickly in some application domains

Question: could you automatically detect when intervention is needed?

The plan for today

Imitation Learning

1. Learning expressive policy distributions
2. Learning from online interventions
3. Time permitting: how to collect demonstrations

} Topic of homework 1!

How to collect demonstrations?

In some domains: People already collect demonstrations that can be recorded
e.g. driving cars, writing text messages

What about robotics?

Kinesthetic teaching



- + easy interface
- human visible in scene

Remote controllers



- ~ interface ease varies,
can have high latency

Puppeteering



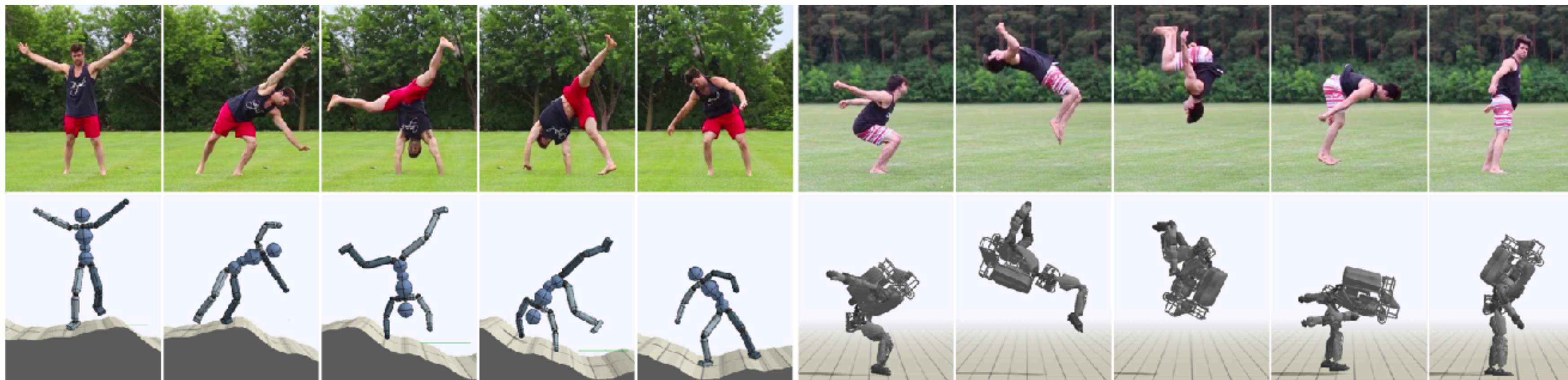
- + easy interface
- requires double hardware

In other domains: It may not be viable to collect demos! (e.g. quadruped robot)

Can robots directly use videos of people, animals?

Embodiment gap: - difference in appearance
 - difference in physical capabilities, degrees of freedom

Hard to directly imitate human & animal data, but can potentially guide exploration.



Peng, Kanazawa, Malik, Abbeel, Levine. SFV: Reinforcement Learning of Physical Skills from Videos. SIGGRAPH Asia 2018.

Summary

Part 1: Train policy to mimic offline demonstrations

- best if policy is an expressive generative model over actions
- algorithm is fully *offline*

often referred to as
behavior cloning (BC)

Part 2: Improve policy using online interventions

- requires interface for human or expert intervention
- algorithm involves running policy *online*

often referred to as
DAgger, or **HG-DAgger**

- + **offline BC**: simple, no need for data from policy (online data can be unsafe, expensive)
- + **DAgger**: possible path to reliable performance, more data-efficient than offline BC
- + no need to define a reward function
- may need **impractically large amount** of data for reliable performance
- doesn't provide a framework for improving on own (from "practicing")

Many successful methods **combine** imitation learning and reinforcement learning!

The plan for today

Imitation Learning

1. Imitation learning basics
2. Learning expressive policy distributions
3. Learning from online interventions
4. Time permitting: how to collect demonstrations

} Topic of homework 1!

Key learning goals:

- how to represent distributions with neural networks
- why expressive distributions matter for imitation learning
- what are compounding errors and how to address them

Next time

Start of *reinforcement learning algorithms*

Course reminders

- Start forming final project groups (survey due Weds April 22)
- Homework 1 out, due Fri April 10
- PyTorch tutorial today at 3:30 pm in Skilling Auditorium